

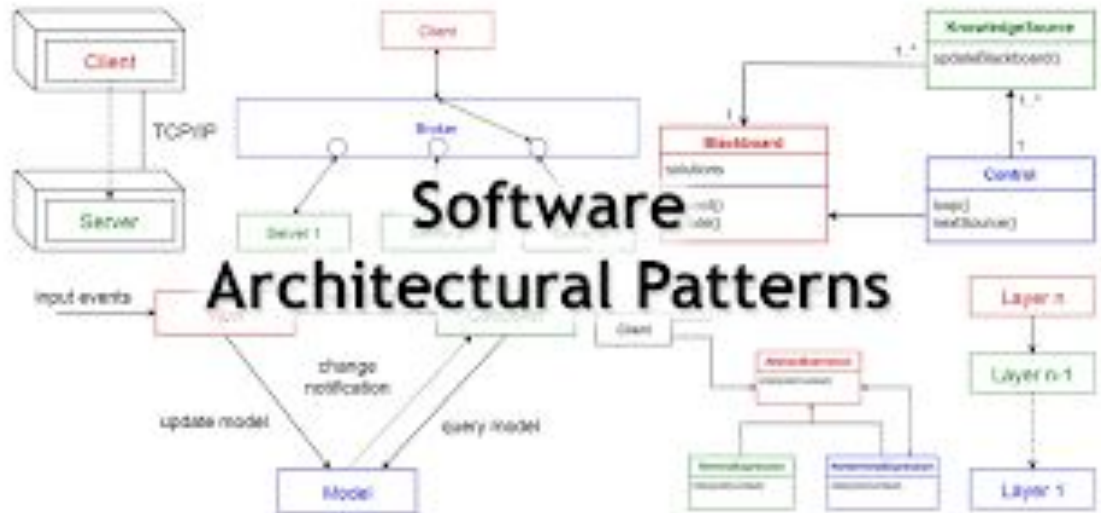
COMS W4156 Advanced Software Engineering (ASE)

November 28, 2023



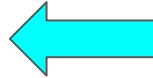
Agenda

1. Revisit software architecture
2. Microservice Architecture
3. ROS middleware architecture



Reprise: Software Architecture

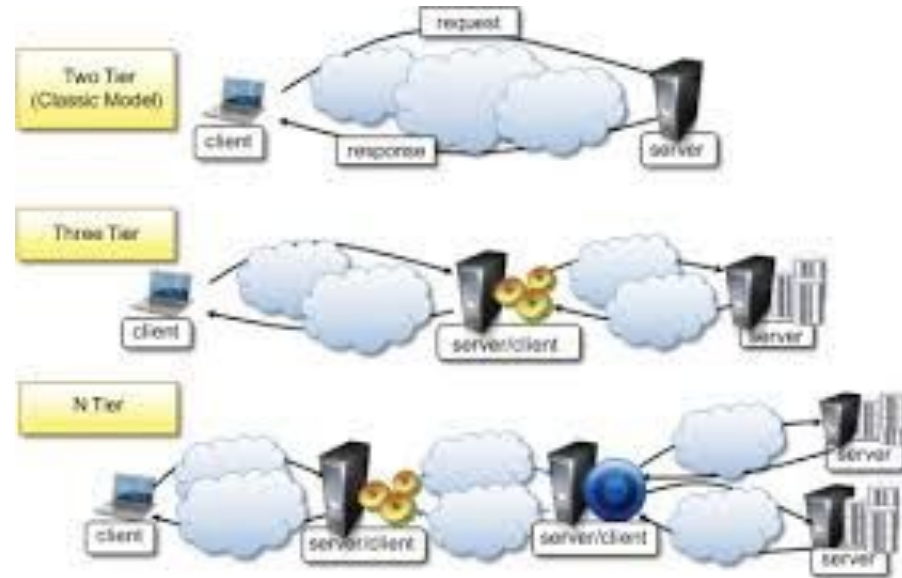
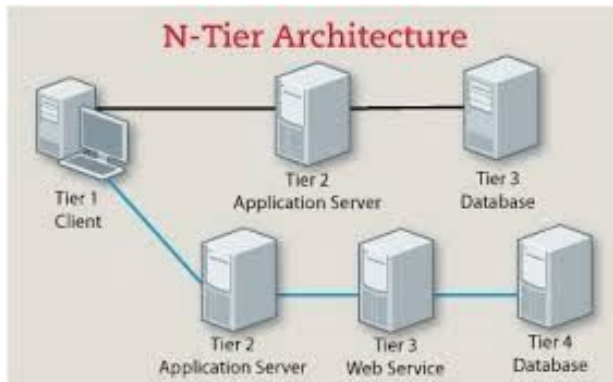
Client-server or N-tier request-response



Broker or gateway mediated request-response

Service-oriented architecture (SOA)

Publish-subscribe or event-driven push or pull



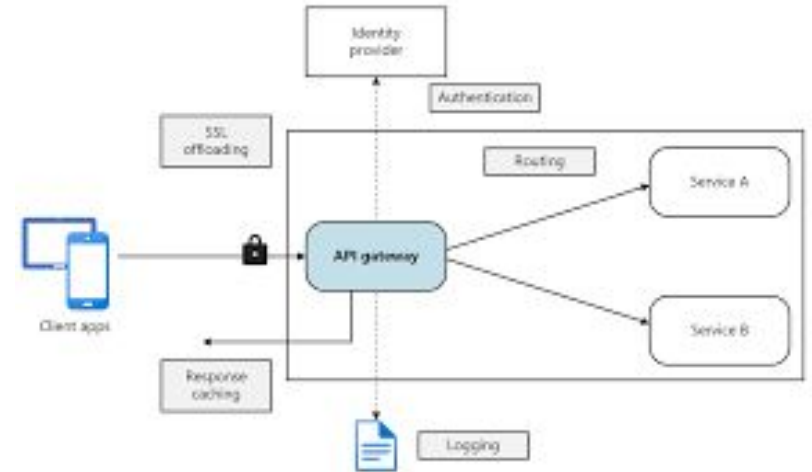
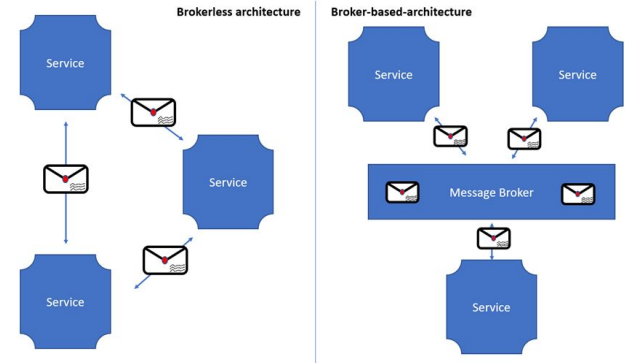
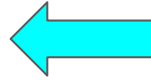
Reprise: Software Architecture

Client-server or N-tier request-response

Broker or gateway mediated request-response

Service-oriented architecture (SOA)

Publish-subscribe or event-driven push or pull



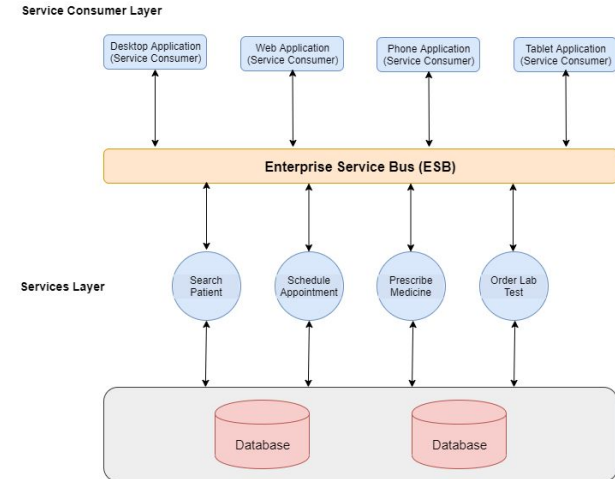
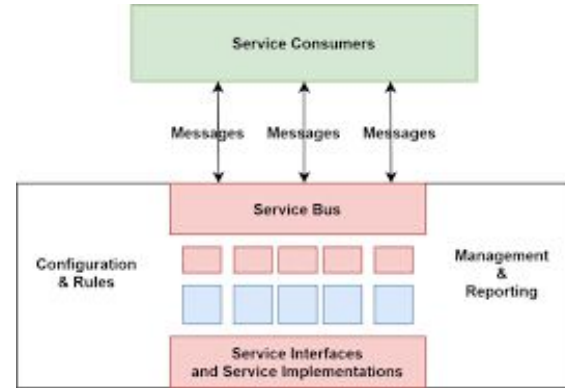
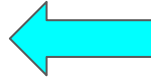
Reprise: Software Architecture

Client-server or N-tier request-response

Broker or gateway mediated request-response

Service-oriented architecture (SOA)

Publish-subscribe or event-driven push or pull



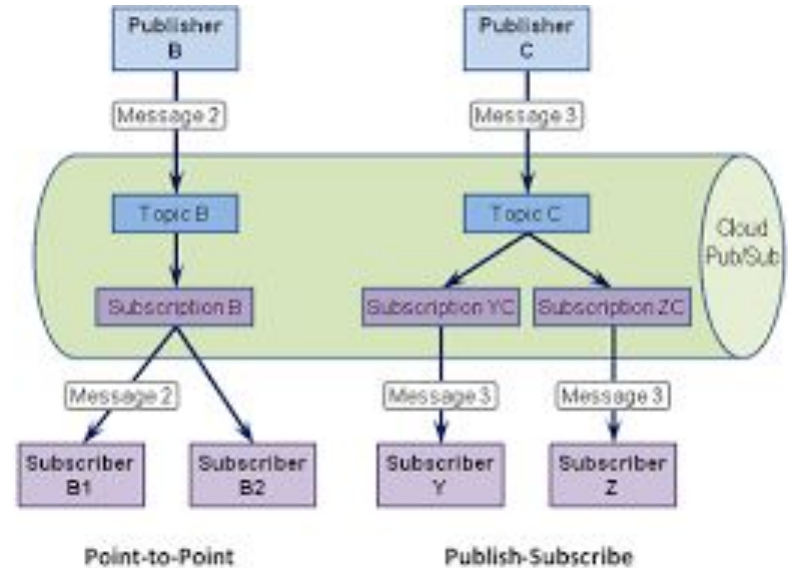
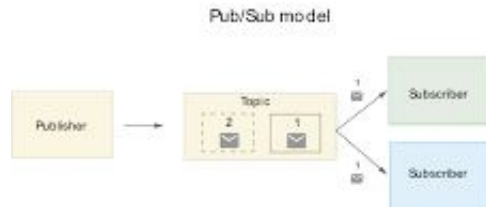
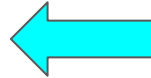
Reprise: Software Architecture

Client-server or N-tier request-response

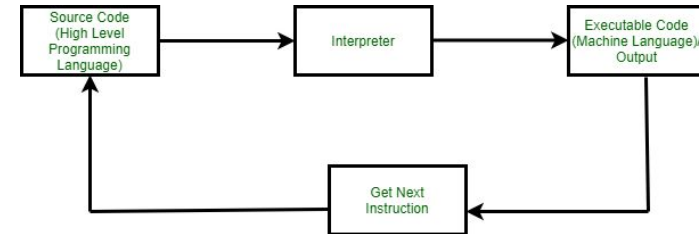
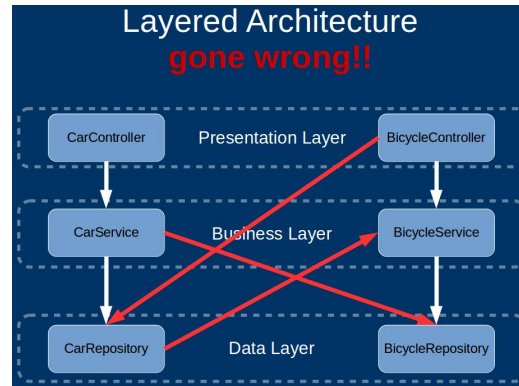
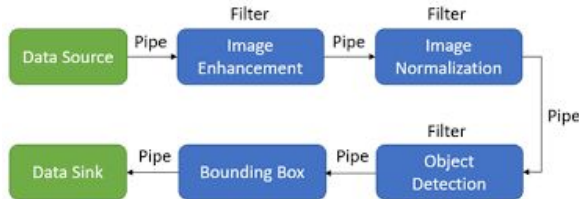
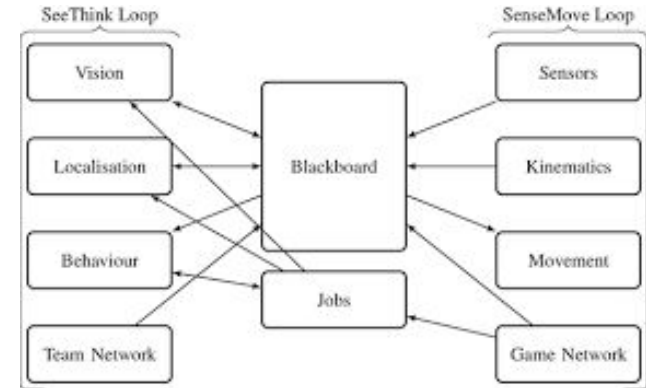
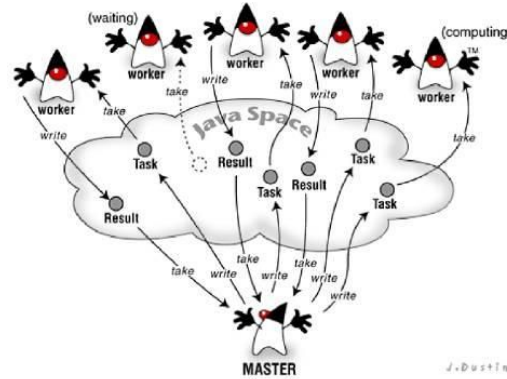
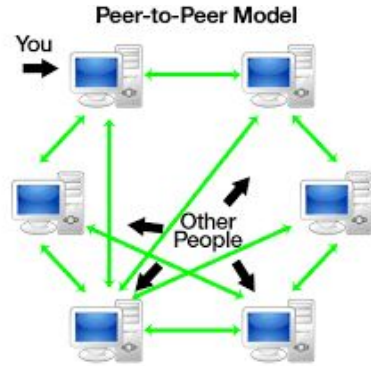
Broker or gateway mediated request-response

Service-oriented architecture (SOA)

Publish-subscribe or event-driven push or pull

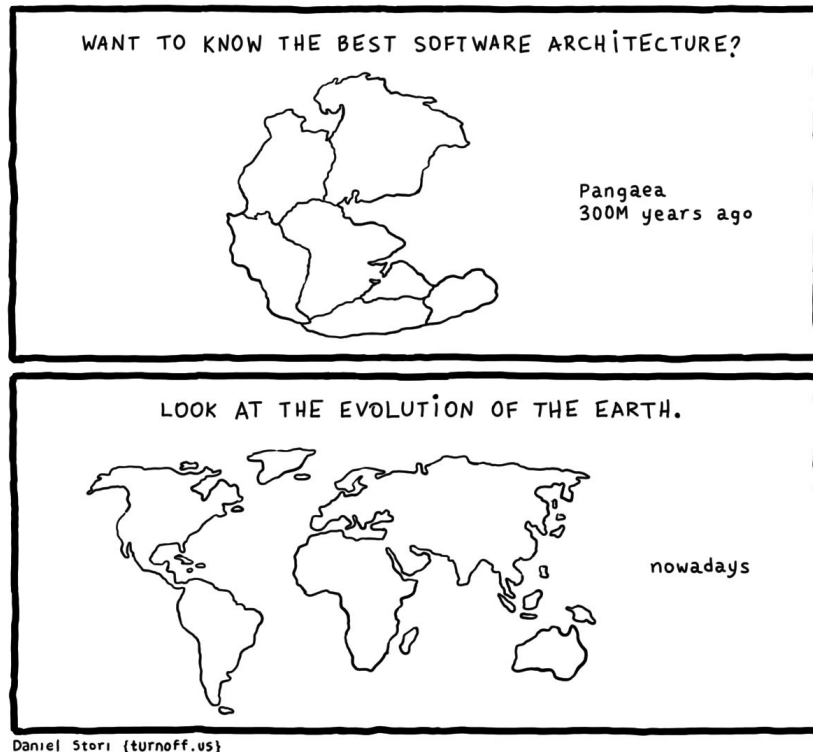


There are many other software architectures, here's a few



Agenda

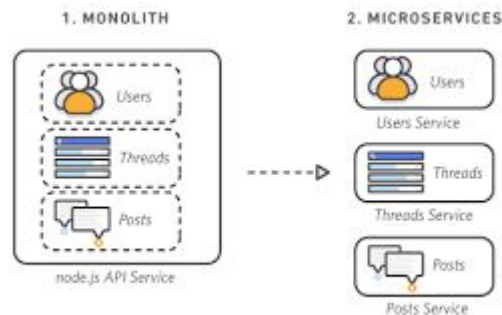
1. Revisit software architecture
2. **Microservice Architecture**
3. ROS middleware architecture



What are Microservices?

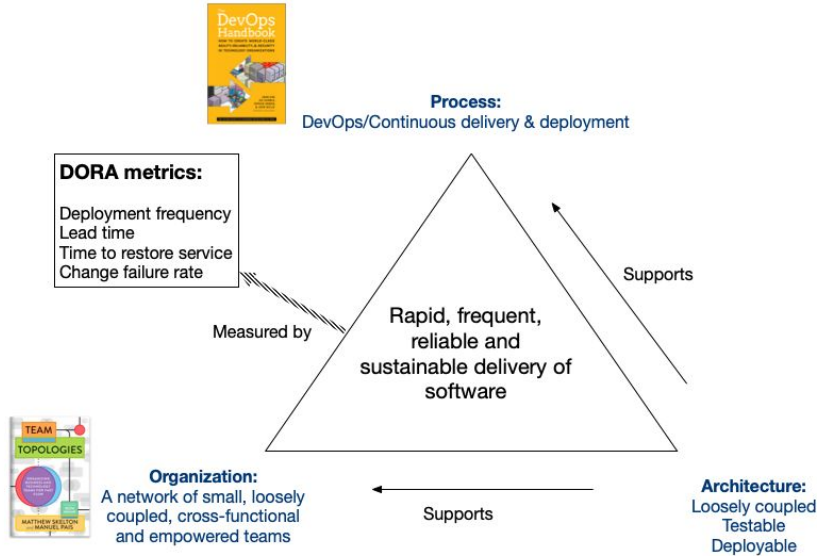
Microservices - also known as [microservice architecture](#) - is an architectural style that structures an application as a collection of services that are:

- Highly maintainable and testable
- Independently deployable
- Loosely coupled
- Organized around business capabilities
- 'Owned' by a small team



Enables an organization to deliver large, complex applications rapidly, frequently, reliably and sustainably

Enabling rapid, frequent, reliable and sustainable software delivery



Rapid, frequent, reliable and sustainable delivery requires 'the success triangle', a combination of:

- Process - DevOps
- Organization - a network of small, loosely coupled, cross-functional teams
- Architecture - a loosely coupled, testable and deployable architecture

Teams work independently most of the time to produce a stream of small, frequent changes that are tested by an automated deployment pipeline and deployed into production

DevOps Research and Assessment ([DORA](#)) metrics

- Deployment frequency - the frequency with which software changes are deployed to production.
- Lead time for changes - the time it takes to go from commit to deploy.
- Time to restore service - the time it takes to restore service after a production incident or outage.
- Change failure rate - the percentage of production deployments that result in a failure or require remediation.



Independently Deployable

The simplistic definition of 'independently deployable' is a service is packaged as a deployable or executable unit.

Examples include: executable JAR file, WAR file, operating system executable, a Docker container image, or a Zip file.

Conversely, a regular JAR file that needs to be packaged with other JARs is not a deployable or executable unit.

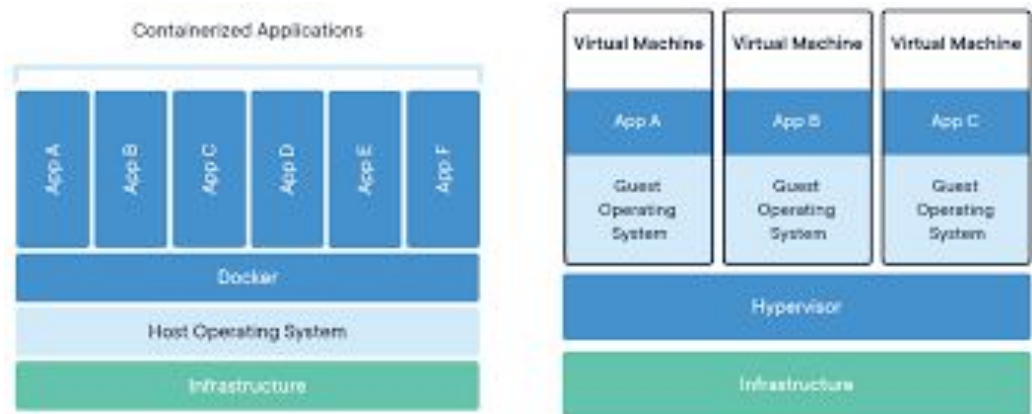


Independently deployable = production-ready after being tested in isolation

A more useful definition of *independently deployable* is a service that is packaged as a deployable or executable unit **and** is production-ready after it has been tested in isolation.

Such a service has its own source code repository, external dependencies, and deployment pipeline.

[Containers](#) are well-suited to
deploying individual microservices.



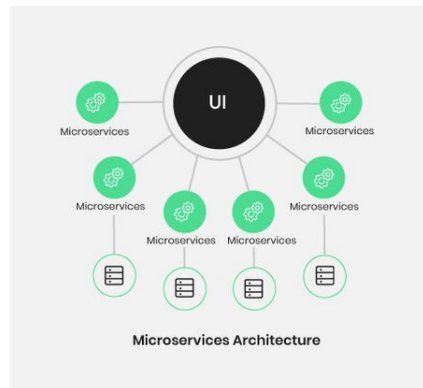
Independently deployable = production-ready after being tested in isolation

The deployment pipeline tests the service in isolation from the rest of the system by using [test doubles](#) for its collaborators.

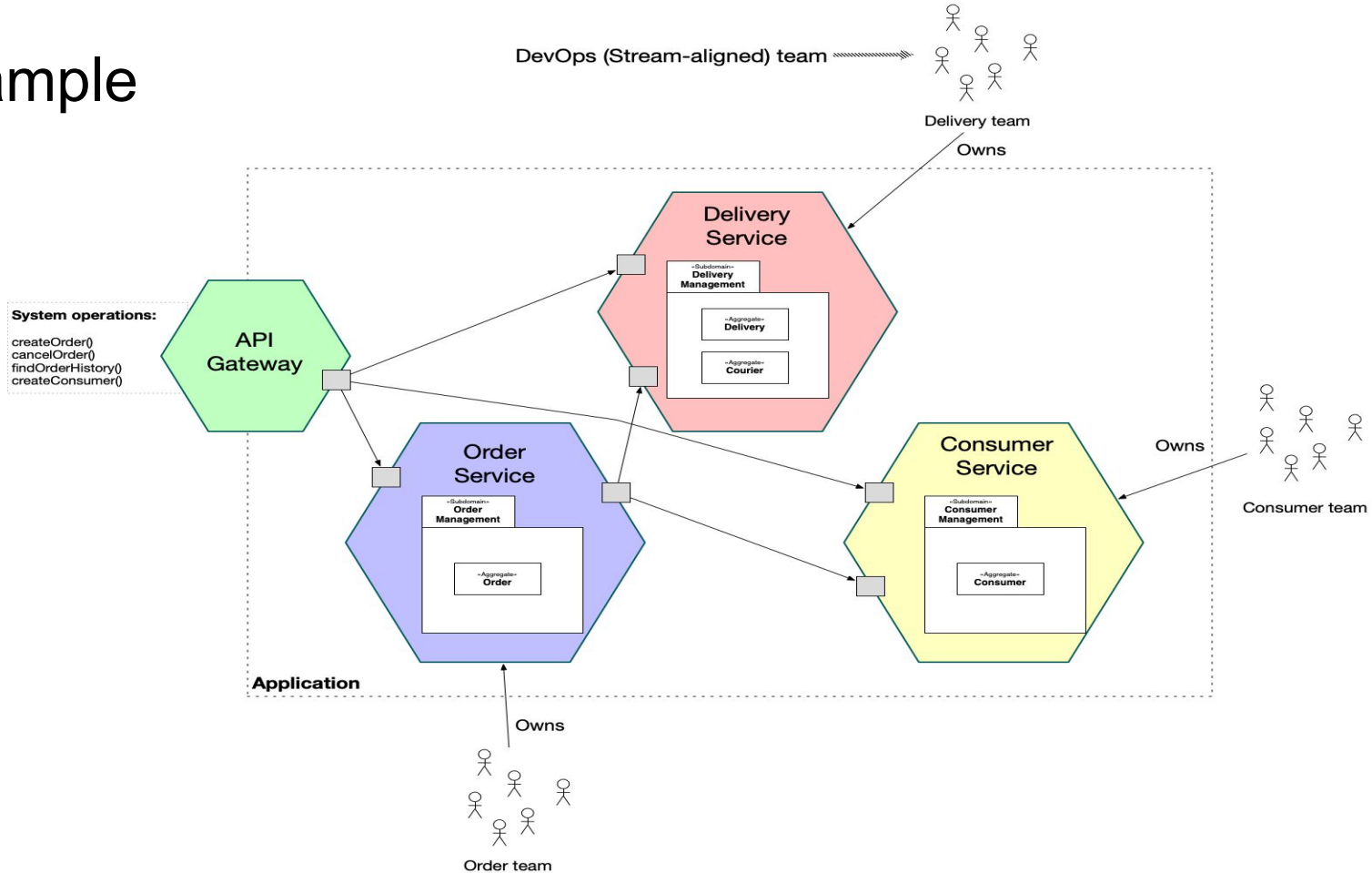
What emerges from the deployment pipeline is a service that can be deployed into production.

If a service needs to be tested with other services to verify that it's production ready then it's not independently deployable.

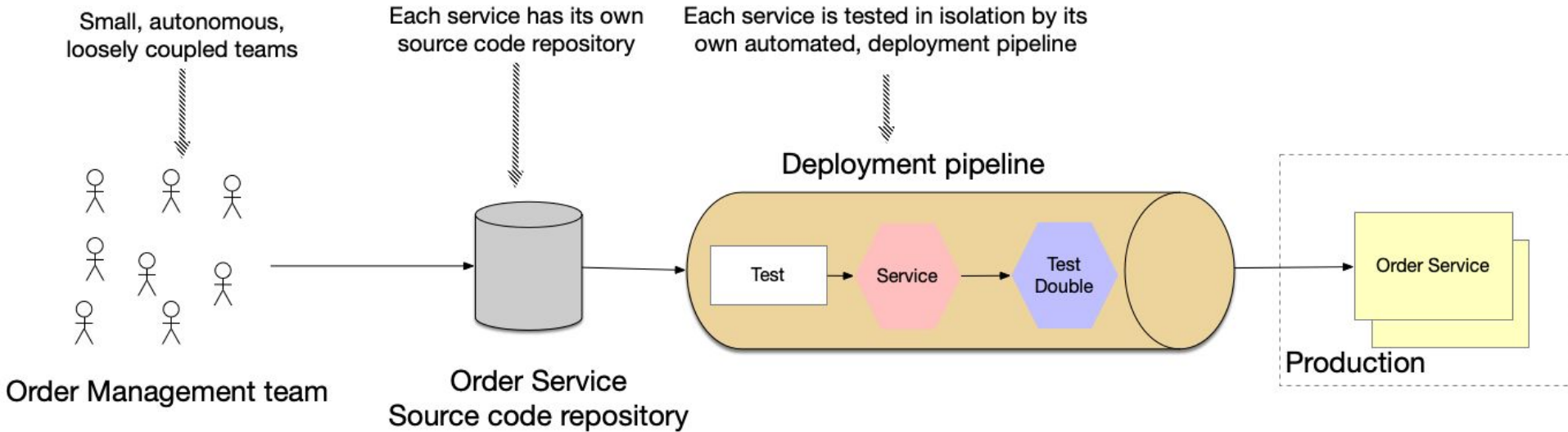
An important benefit of independent deployment is it accelerates the deployment pipeline, eliminating the need for slow, brittle, and complex end-to-end tests of multiple services across coordinating teams.



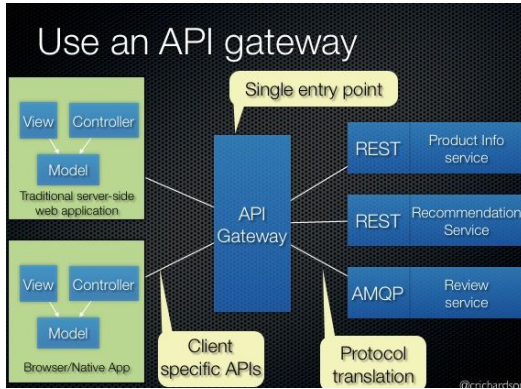
Example



Independent Deployment Pipelines

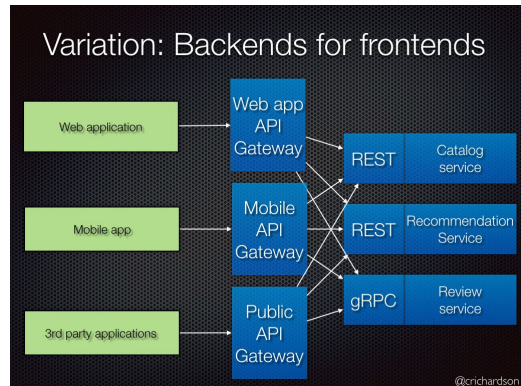


What is an API gateway?



An API gateway is an [API management](#) tool that sits between a client and a collection of backend services.

An API gateway acts as a reverse proxy to accept all application programming interface ([API](#)) calls, aggregate the various services required to fulfill them, and return the appropriate result.



Some common functions of API management include authentication, routing, rate limiting, billing, monitoring, analytics, policies, alerts, and security.

Rethinking user acceptance tests



Microservices allow a large application to be separated into smaller independent parts, with each part having its own realm of responsibility.

To serve a single user request, a microservices-based application can call on many internal microservices to compose its response.

One potential obstacle to deploying services independently are *system-level* [user acceptance tests](#).

Acceptance tests are usually written from the perspective of a user and often span multiple services - remember that the multiple microservices are all part of one application from the end-user perspective.

A straightforward implementation of such tests would require testing multiple services together - but for services to independently deployable, system-level user acceptance tests must be replaced with service-level user acceptance tests.

? How can system UAT be distributed to service UAT ?

Types of User Acceptance Testing

01 Alpha testing
Alpha testing is executed by QA team by imitating the real users' behavior.

02 Beta testing
Beta testing is conducted by target users to assess product performance.

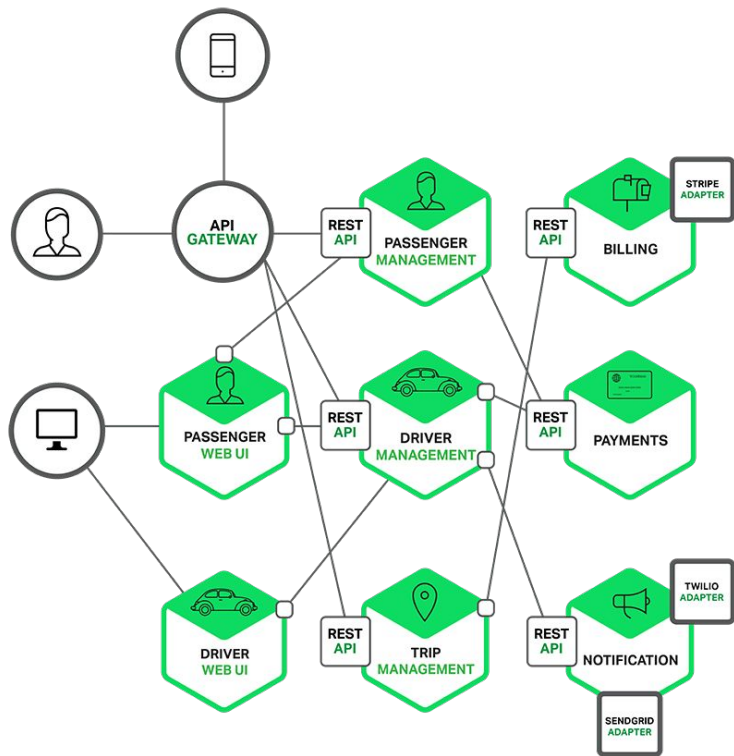
03 RAT
Regulation acceptance testing ensures compliance of the software with the government norms.

04 CAT
Contract acceptance testing verifies if a product meets stated in the contract criteria.

05 BAT
Business acceptance testing checks if a product complies with business requirements.

06 OAT
Operational acceptance testing checks whether an app provides a proper workflow.

An independently deployable service requires a well-defined specification



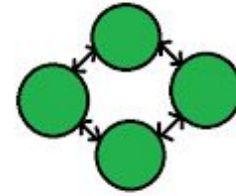
For a service to be testable in isolation it needs to have (a) a well-defined specification and (b) a test suite that verifies that its behavior conforms to that specification.

Services must also collaborate through stable APIs in order avoid regularly changing in lockstep (loose vs. tight coupling).

Loosely Coupled



Tight coupling:
1. More interdependency
2. More coordination
3. More information flow

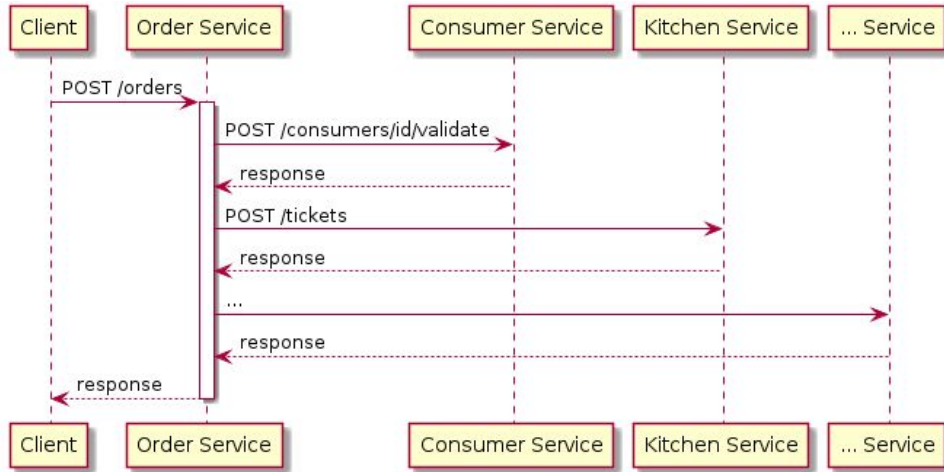


Loose coupling:
1. Less interdependency
2. Less coordination
3. Less information flow

There's two different types of coupling:

- [runtime coupling](#) - influences availability, runtime coupling between services is the degree to which the availability of one service is affected by the availability of another service (the availability of an operation implemented by one service is affected by the availability of another service)
- [design-time coupling](#) - influences development velocity, the degree of design-time coupling between a pair of software elements (whether classes or services) is the likelihood that they need to change together for the same reason

Runtime coupling and availability



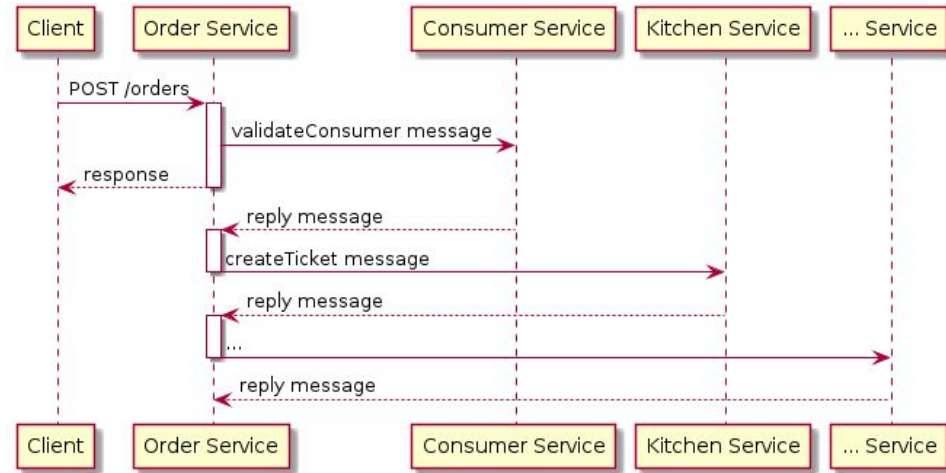
For example, let's imagine that the `createOrder()` system operation is implemented by an `HTTP POST /orders` endpoint in the `Order Service`. The `Order Service` handles the `HTTP POST` by invoking other services, waiting for them to response, and then sending a response to its client.

In this design, the `Order Service` cannot respond to the `POST` request until the other services respond to it. The `Order Service` (or `createOrder()` operation) is said to be runtime coupled to those other services. As a result, the availability of the `createOrder()` operation is reduced since all services must be available.

Runtime coupling and availability

Another way to reduce runtime coupling is to design self-contained services. A self-contained service responds to a synchronous request with a partial outcome and then asynchronously completes the operation.

For example, the **Order Service** could respond to the **HTTP POST /orders** request with a **202 Accepted** response and then initiate a **Create Order Saga** to complete the operation. This approach improves the availability of the **Order Service**. The drawback is that it makes the client more complex since it must be able to handle partial outcomes and somehow determine the final outcome of the operation.

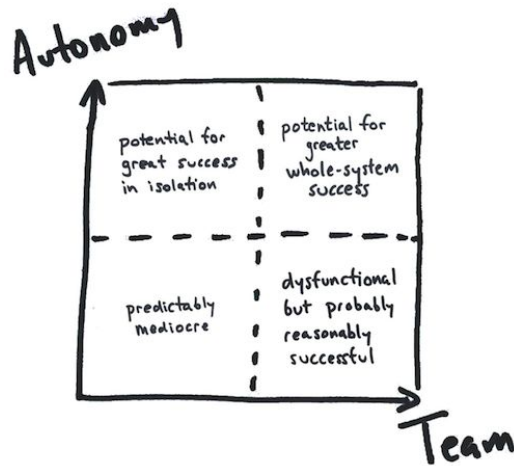


Design-time coupling reduces development velocity

If two services are loosely coupled, then a change to one service rarely requires a change to the other service. However, if two services are tightly coupled, then a change to one service often requires a change to the other service. Lockstep changes are expensive since they typically involve breaking API changes.

For example, let's imagine that the **Order Service** and **Customer Service** are tightly coupled. Each time a breaking change needs to be made to **Customer Service** the sequence of steps is:

1. Change the **Customer Service** to add a new major version of its API. The service must implement both the old and new version of its APIs until all clients have been migrated over.
2. Migrate the **Order Service** to the new API version
3. Remove the old API version from the **Customer Service**

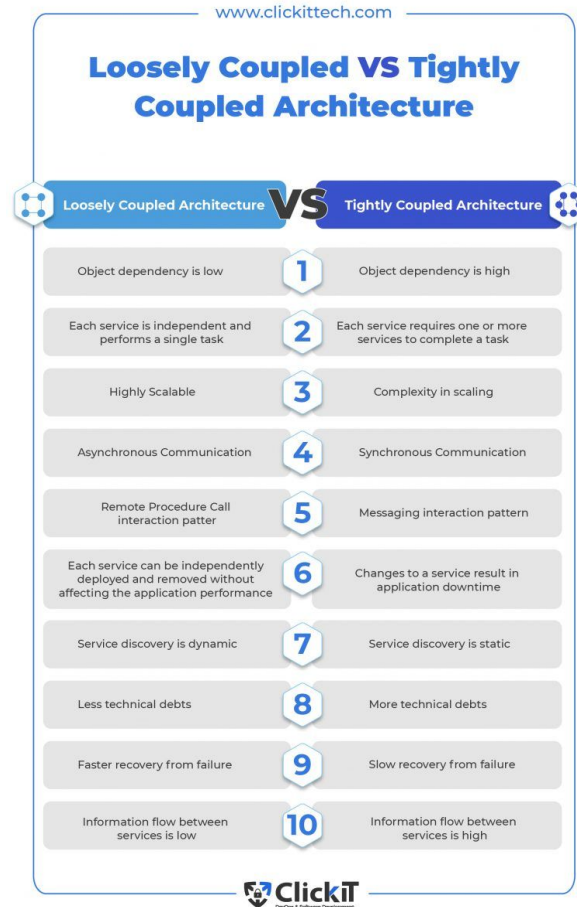


What's even worse, is when the services are owned by different teams that must coordinate the changes, undermining team autonomy.

Reducing design-time coupling

There are two different ways to minimize design-time coupling between services:

- design subdomains to be loosely coupled - loosely coupled subdomains can be packaged as different services. Loose design-time coupling is usually achieved by each subdomain having a stable API that encapsulates its implementation.
- package subdomains that are tightly coupled in the same service - If two subdomains are tightly coupled, then packaging them together in the same service will avoid design-time coupling between services.



Migrating from monolithic to microservices

Most presentations of microservices assume the existing codebase is monolithic, as opposed to designing as microservices from the beginning

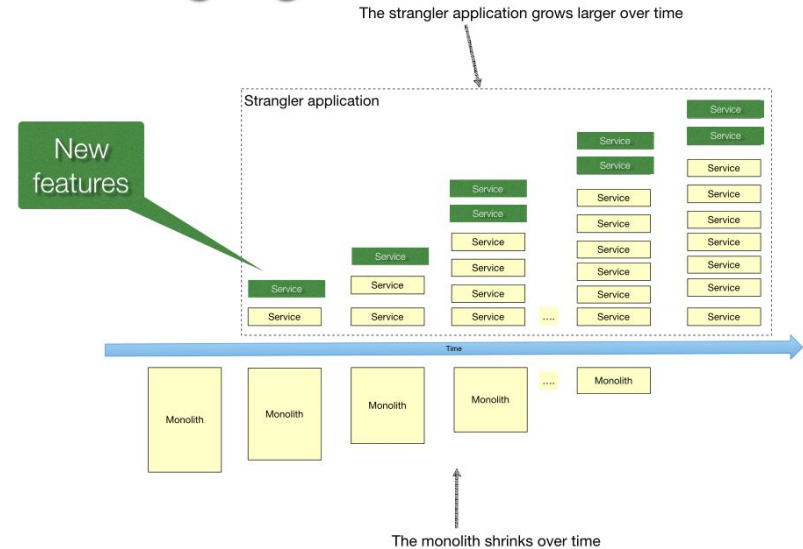
Starting with monolithic makes sense for small teams who seek a quick launch (or [angel investors!](#))

One commonly suggested approach to migration is gradually 'strangling' the monolith as existing features change and new features are added - analogous to gradually replacing test doubles during integration testing vs. 'big bang' - but you still need to add or split teams



Designing a microservices application from scratch may be easier if you know how to design services

Strangling the monolith



Agenda

1. Revisit software architecture
2. Microservice Architecture
3. **ROS middleware architecture**



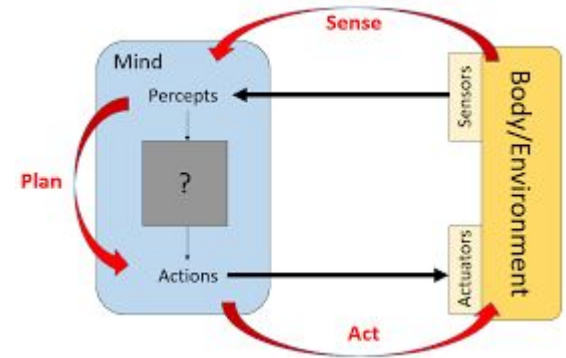
ROS = Robot Operating System

Not an actual operating system like Linux
(ROS typically runs on top of real-time variants of Linux.)

ROS is an open-source framework and ecosystem for robot programming, offering a package management system with pre-existing modules and resources (for navigation, mapping, manipulation, computer vision, machine learning, etc.) that can be utilized to build robot applications more efficiently than starting from scratch.

Perhaps most significantly, ROS facilitates inter-process communication - instead of creating a single program to handle all the sensors and actuators, it is much more efficient to modularize the system and communicate via APIs and 'plumbing'

Analogous to microservices for business applications

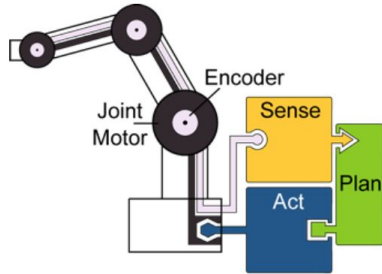


$$\text{ROS} = \text{plumbing} + \text{tools} + \text{capabilities} + \text{community}$$

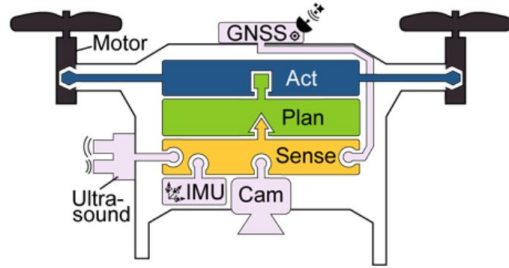
The equation uses icons: a grid of dots for ROS, a pipe for plumbing, a gear for tools, a person with a checkmark for capabilities, and puzzle pieces for community.

What Kinds of Systems Use ROS?

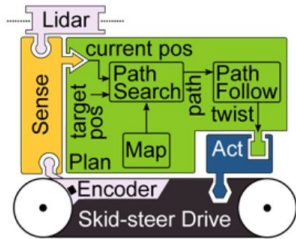
Figure 1.2. An example of different robot systems and how they can be structured in a common way. The diagrams reflect the relative importance of each part through its size and degree of detail.



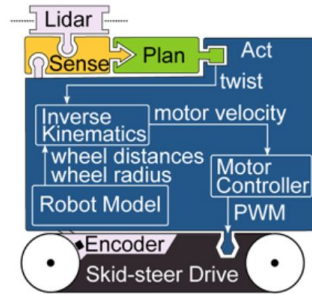
(a) Industrial Manipulator



(b) Quadcopter



(c) Mobile Robot,
focus on planning



(d) Mobile Robot,
focus on actuation

Robot programming is essentially the process of giving instructions to a robot so that it can perform tasks.

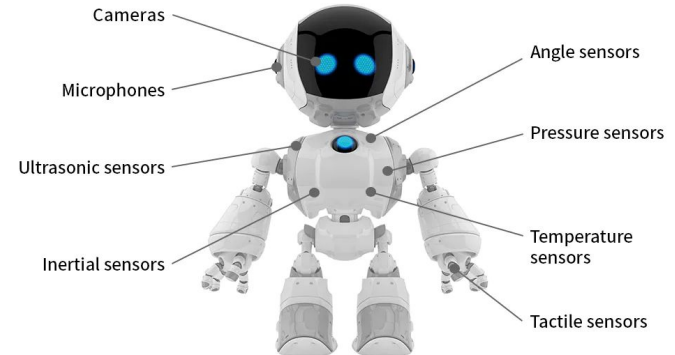
Performing tasks can be broken down into three steps: sense, plan, and act.

The computer inside the robot acts as its brain (plan) and is connected to sensors and actuators - the computer processes data from the sensors (sense) and sends control signals to the actuators (act).

What does a sensor sense?

Sensors are devices that measure quantities, which the robot can use to achieve given tasks. May be 'external' (measuring environment) or 'internal' (measuring the robot itself).

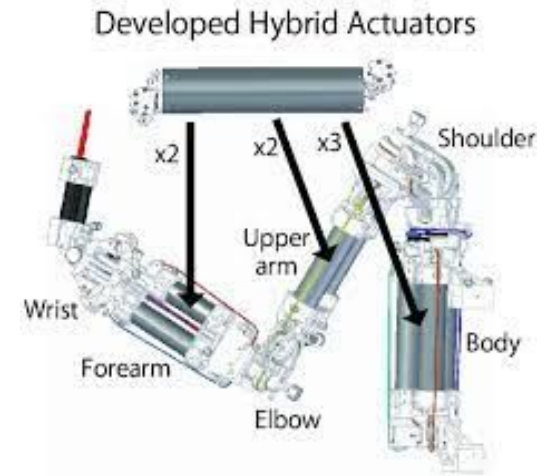
- How far away and where is an object like another robot, a light, a wall?
- Where is the robot and in what direction is it going?
- At what speed is the robot moving?
- How far it has it travelled?
- If the robot is on a hill, at what angle is it?
- If the robot is rotating, by how much has it rotated?
- How much power has it consumed?
- What information is being communicated to the robot?



What acts does an actuator do?

An actuator is a device that causes something to happen.

- An actuator is needed to make the robots wheels turn.
- Or the joints of a robot arm to rotate.
- Or for a robot gripper to open or close.
- A light being turned on to indicate the robot is working.
- Sound is being emitted – so the actuator is a loudspeaker.
- Communication in some form – so a transmitter is needed.
- An LCD (Liquid Crystal Display) screen for showing pictures and other data.



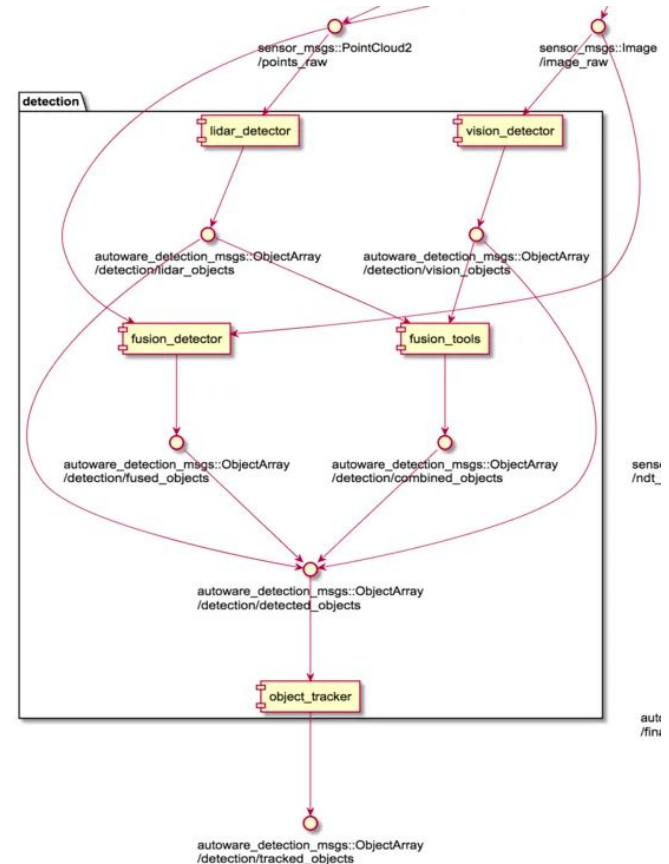
ROS Programs are written as 'computational graphs'

A [ROS graph](#) is a network of nodes that work together to solve problems.

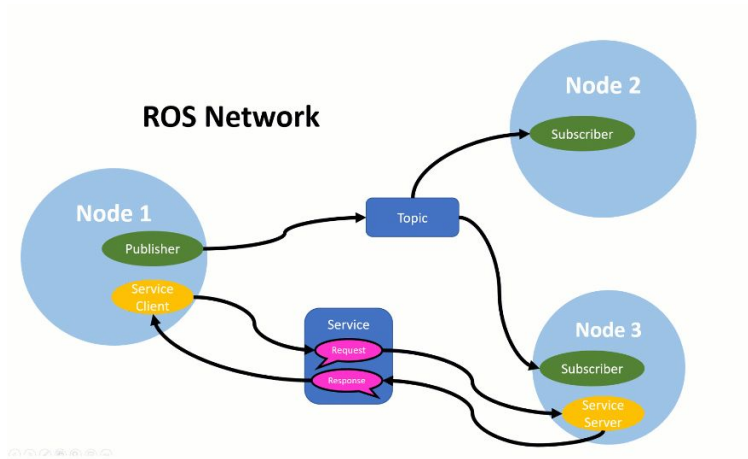
Each '[node](#)' is a process that does some kind of computation, and the 'edges' between the nodes are the ways that the nodes share data with each other.

By connecting the nodes in a computational graph through '[discovery](#)', you make a distributed system in which each node does a specific task and talks to other nodes using publish-subscribe and/or request-response over [DDS](#) (Data Distribution Service).

Nodes corresponding to sensors and actuators use adapters (wrappers) to fit into DDS paradigm



ROS Publish/Subscribe



A node that sends messages to a certain topic is called a publisher and a node that receives messages from a certain topic is called a subscriber.

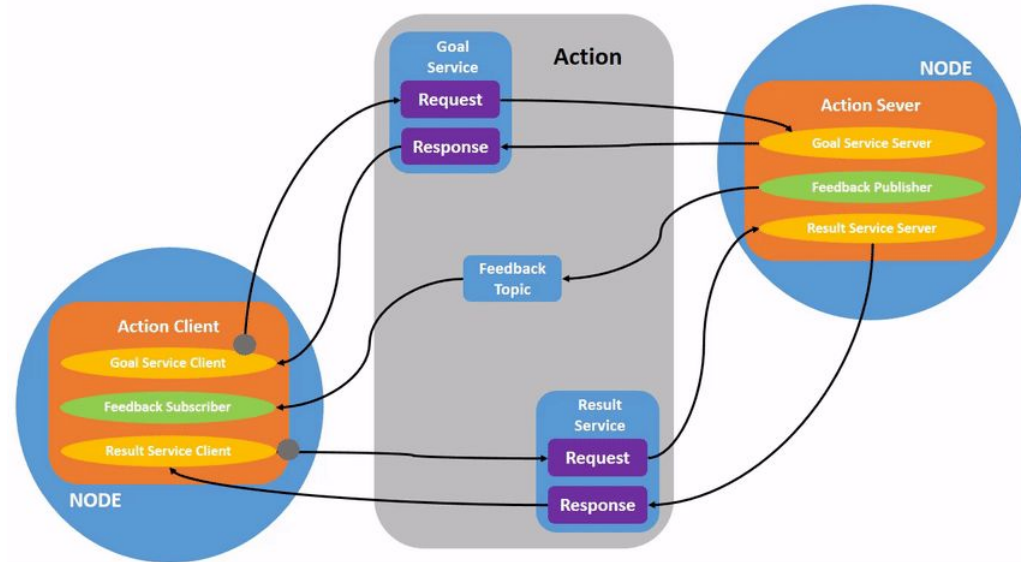
A node creates a publisher object and uses it to send a message to a topic.

A node creates a subscriber object and tells it which topic it wants to subscribe to so it can receive messages. The subscriber will then get any messages that are posted to that topic.

ROS Request-Response

Nodes can also provide [services](#) as a way for other nodes to request specific computation or data and receive a response.

Services are implemented using a request-response messaging pattern, essentially RPC, where a node that wants to request a computation or data (the *client*) sends a request message to another node (the *server*), which performs the computation or retrieves the data and sends a response message back to the client.



[Actions](#) allow for long-running service calls with intermediate results reported via topics.

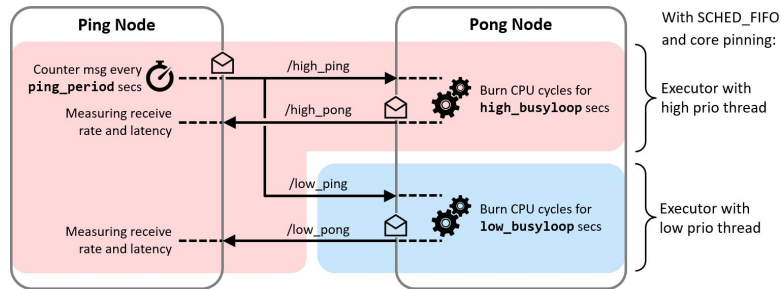
ROS Executor

Many ROS applications require predictable execution within real-time limits.

For example, a *Timer* can schedule a callback function to be executed at a specific time or at regular intervals.

An [Executor](#) manages scheduling and execution of ROS nodes and resources, but has limitations such as no mechanisms to enforce a certain execution order of callbacks within a node - so a low priority task can indirectly pre-empt a high priority task (priority inversion).

[Researchers have proposed various improvements](#) related to real-time performance constraints.

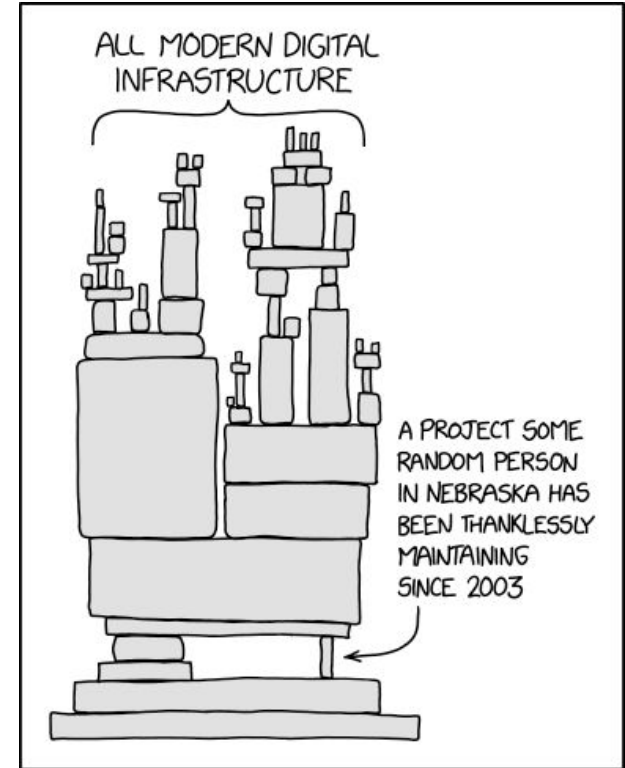


Other Drawbacks and Limitations

Complexity: ROS is comprehensive and powerful, but this can make it relatively complex to learn and use. It may require a significant investment of time and effort to become proficient.

Dependency on open-source software: ROS is built on top of a large collection of open-source software libraries and tools, which can make it dependent on the availability and stability of this software - a concern for projects that need to ensure long-term support and maintenance.

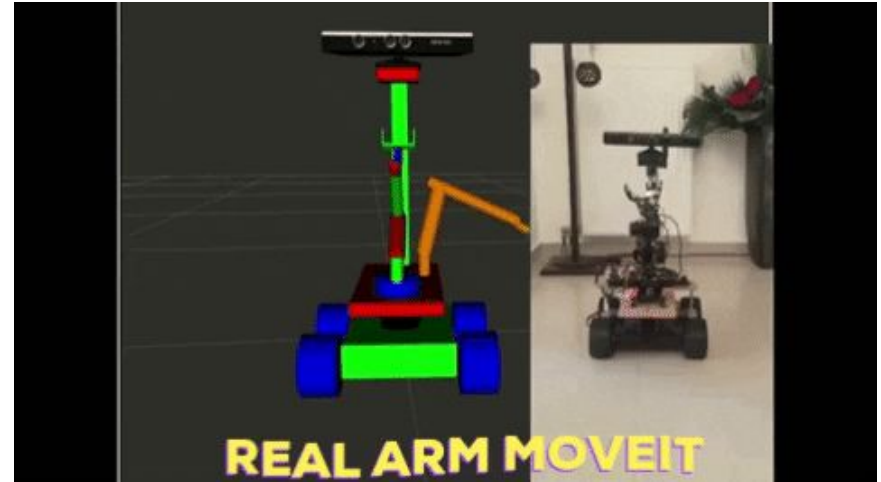
Limited support for proprietary platforms: An issue for projects that need to integrate with proprietary or legacy systems or technologies.



*ROS2, the robot's friend,
A platform on which they depend,
For communication, coordination, and control,
It helps them achieve their goals.*

*With DDS at its core,
ROS2 helps robots explore, New worlds, new
tasks, and new possibilities, It's a platform that
facilitates and facilitates ease.*

*From turtles to drones, and everything in between,
ROS2 helps robots work, learn, and dream,
It's a powerful tool, a powerful ally,
Helping robots reach their highest potential, and
never say goodbye.*



DDS and much of this material is specific to ROS2. [ROS1 is similar but different](#), e.g., ROS1 is centralized with master/worker paradigm whereas ROS2 is (conceptually) distributed.

Remaining Assignments

- [Second Iteration](#) due this Thursday, November 30, by 11:59pm
- [Second Iteration Demo](#) due next Thursday, December 7, by 11:59pm

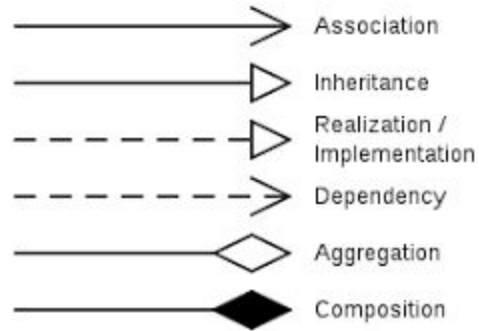
Second Assessment will be available Friday December 8 at 12:01am and due Monday December 11 at 11:59pm (4 days minus two minutes)

[Demo Day](#) is Monday, December 18, 10am-4pm, signup sheet is [here](#)



Thursday

UML basics



Ask Me Anything

**If you have any last-minute
questions about your team projects,
now would be a really good time to ask!**